MDL Exam Cheatsheet

Chapters 8–11 | Classification, Trees, KNN, Ensembles, Clustering

Compiled from Study Guide + Handwritten Notes (Dr. Sujit)

1. Classification

Setup

- Input: $x \in \mathbb{R}^d$, $x = (x_1, \ldots, x_d)$
- Labels: $y \in [k] = \{1, 2, \ldots, k\}$. If $k = 2$: binary classification
- **Classifier:** $f : X \to Y$. **Scoring fn:** $h : X \to \mathbb{R}$

Confusion Matrix

	Pred P	**Pred N**
True P	TP	FN
True N	FP	TN

Rows = actual, Columns = predicted (from notes)

All Metrics

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{1}$$

$$\text{Recall (TPR)} = \frac{TP}{TP + FN} = \frac{TP}{P} \tag{2}$$

$$\text{Precision (PPV)} = \frac{TP}{TP + FP} \tag{3}$$

$$F_1 = \frac{2}{\frac{1}{\text{Recall}} + \frac{1}{\text{Prec}}} = \frac{2TP}{2TP + FP + FN} \tag{4}$$

$$\text{FPR} = \frac{FP}{FP + TN} = \frac{FP}{N} \tag{5}$$

$$\text{FNR} = \frac{FN}{FN + TP} = \frac{FN}{P} \tag{6}$$

$$\text{TNR (Spec)} = \frac{TN}{TN + FP} = \frac{TN}{N} \tag{7}$$

Exam Tip

Accuracy is **misleading for imbalanced data**. A classifier that always predicts the majority class gets high accuracy but 0 recall on the minority. **Use F1 instead.**

2. Bayes Classifier & Naïve Bayes

Bayes Theorem

$$P(A \mid B) = \frac{P(B \mid A) \cdot P(A)}{P(B)} = \frac{P(B \mid A) \cdot P(A)}{\sum_{A'} P(B \mid A')P(A')} \tag{8}$$

Bayes Classifier

- **Prior:** $P(Y{=}k)$ — class probability before seeing data
- **Likelihood:** $P(X{=}x \mid Y{=}k)$ — feature prob given class
- **Posterior:** $P(Y{=}k \mid X{=}x)$ — what we want
- **Evidence:** $P(X{=}x)$ — normalizer, **does NOT affect decision**

$$P(Y{=}k \mid X{=}x) = \frac{\overbrace{P(X{=}x \mid Y{=}k)}^{\text{Likelihood}} \cdot \overbrace{P(Y{=}k)}^{\text{Prior}}}{\underbrace{P(X{=}x)}_{\text{Evidence}}} \tag{9}$$

$$\hat{y} = \arg\max_k \; P(X{=}x \mid Y{=}k) \cdot P(Y{=}k) \tag{10}$$

Optimal classifier: minimizes error rate.

Curse of Dimensionality

- d binary features $\Rightarrow 2^d$ distributions per class
- $d = 20$: $2^{20} \approx 10^6$ parameters per class — **impractical!**

Naïve Bayes: The Fix

Key Assumption

Features are **conditionally independent given the class**:

$$P(X{=}x \mid Y{=}k) = \prod_{j=1}^{d} P(X_j{=}x_j \mid Y{=}k)$$

Naïve Bayes Classification Rule:

$$\hat{y} = \arg\max_{i \in [k]} \; P(Y{=}i) \prod_{j=1}^{d} P(X_j{=}x_j \mid Y{=}i) \tag{11}$$

- Reduces parameters: $2^d \to 2d$ per class (binary features)
- Independence often violated, but **still works** — only the *ranking* of posteriors matters, not exact values
- Great for **text classification** (spam, sentiment)

Generative vs Discriminative

	Generative	**Discriminative**
Models	$P(X, Y)$ joint	$P(Y \mid X)$ directly
Generate?	Yes	No
Examples	NB, HMM, GMM	LogReg, SVM, DT

3. Decision Trees

Structure

- **Internal nodes:** test a feature
- **Branches:** feature values
- **Leaf nodes:** assign class labels
- **Interpretable:** explicit if-then rules

Entropy (Impurity Measure)

$$H(S) = -\sum_{i=1}^{k} p_i \log_2 p_i \tag{12}$$

p_i = proportion of class i in set S.

Properties:

- $H = 0$: pure node (all one class)
- $H = 1$: max uncertainty (binary 50/50)
- Convention: $0 \cdot \log_2 0 = 0$
- From notes: 2 messages = 1 bit, 4 messages = 2 bits

Example: 9 positive, 5 negative (n=14):

$$H = -\tfrac{9}{14}\log_2\tfrac{9}{14} - \tfrac{5}{14}\log_2\tfrac{5}{14} = 0.940 \text{ bits}$$

Information Gain (ID3)

$$IG(S,A) = H(S) - \sum_{t\in T}\frac{|S_t|}{|S|}\cdot H(S_t) \tag{13}$$

T = values of attribute A; S_t = subset where A=t.
Rule: Pick attribute with **highest** IG at each node.

ID3 Algorithm

1. If all samples same class $c \Rightarrow$ return Leaf(c)
2. If no features left $\Rightarrow$ return Leaf(majority class)
3. $A^* = \arg\max_A IG(S,A)$
4. Create node for A^*; for each value v:
   - $S_v = \{x \in S \mid x.A^* = v\}$
   - If $S_v = \emptyset$: Leaf(majority of S)
   - Else: recurse on $(S_v, \text{Features} \setminus \{A^*\})$

CART (Binary Splits + Gini)

ID3 vs CART

ID3: multi-way splits + entropy/IG.
CART: only 2 children (binary splits) + Gini index.

Gini Index:

$$\text{Gini}(S) = 1 - \sum_{i=1}^{k} p_i^2 = \sum_i p_i(1-p_i) \tag{14}$$

- Gini = 0 for pure node; no logarithm needed (faster)
- Gini and entropy produce **similar trees** in practice
- From notes: Gini $= 2\sum p_L(1-p_L)$

Overfitting

From Notes

"Small change in data leads to different tree."

- **Pre-pruning:** limit max depth, min samples/leaf, min IG
- **Post-pruning:** grow full tree, remove unhelpful branches
- **Fix:** use **ensembles** (bagging/boosting) to stabilize

4. K-Nearest Neighbors (KNN)

Algorithm

Given $D = \{(x_1,y_1),\ldots,(x_n,y_n)\}$ and query x:

1. Compute distance from x to every training point
2. Sort: $|x_{\sigma(1)} - x| \le |x_{\sigma(2)} - x| \le \cdots$
3. Find the K nearest neighbors
4. $\hat{y}$ = **majority vote** among K neighbors

Lazy Learner

KNN has **no training phase**. All computation at prediction time.

Distance Metrics

$$d_{\text{Euclid}} = \sqrt{\sum_{j=1}^{d}(x_j - x'_j)^2} \quad \text{(continuous)} \tag{15}$$

$$d_{\text{Manhattan}} = \sum_{j=1}^{d}|x_j - x'_j| \tag{16}$$

$$d_{\text{Hamming}} = \sum_{j=1}^{d}\mathbb{1}[x_j \neq x'_j] \quad \text{(discrete)} \tag{17}$$

$$d_{\text{Minkowski}} = \Big(\sum_{j=1}^{d}|x_j - x'_j|^p\Big)^{1/p} \tag{18}$$

p=1: Manhattan. p=2: Euclidean.

Valid Distance Properties

$d : X \times X \to \mathbb{R}_+$:

1. $d(x,y) \ge 0$ (non-negativity)
2. $d(x,y) = 0 \iff x = y$; $d(x,y) = d(y,x)$ (identity + symmetry)
3. $d(x,y) + d(y,z) \ge d(x,z)$ (triangle inequality)

Choosing K

- **Small** K (e.g. 1): low bias, high variance $\Rightarrow$ overfitting
- **Large** K: high bias, low variance $\Rightarrow$ underfitting
- Choose via **cross-validation**

Key Result (Notes)

As $n \to \infty$, 1-NN error $\le$ **2** $\times$ **Bayes optimal error**.

Weaknesses

- **Curse of dimensionality:** all points become equidistant in high-d
- **Needs scaling:** use Z-score $(x-\mu)/\sigma$
- **Slow prediction:** $O(n \cdot d)$ per query

5. Ensemble Learning

From Notes: "Combining Classifiers"

Instead of one classifier, **combine multiple** for better performance.

Bagging (Bootstrap Aggregating)

Idea: Train on **different random subsets**, combine by **majority vote**.

Algorithm:

1. Create B bootstrap samples (sample n points **with replacement**)
2. Train a separate classifier on each sample
3. Prediction: each classifier votes $\Rightarrow$ **majority wins**

Properties:

- Models trained **independently** (parallelizable)
- All models have **equal weight**
- Reduces **VARIANCE**
- Best for high-variance models (deep trees)
- Example: **Random Forest** = Bagging + Decision Trees

Boosting

Idea: Train **sequentially**, each focusing on **previous mistakes**. Combine by **weighted vote**.

AdaBoost Algorithm:

1. Init: all sample weights $w_i = 1/n$
2. For $t = 1, \ldots, T$:
   - Train h_t on weighted data
   - Weighted error: $\epsilon_t = \sum_{i:\, h_t(x_i) \neq y_i} w_i$
   - Classifier weight:

$$\alpha_t = \tfrac{1}{2} \ln\left(\tfrac{1-\epsilon_t}{\epsilon_t}\right)$$

     (better classifiers $\Rightarrow$ higher α)
   - **Increase** weights of misclassified, **decrease** correct
   - Normalize weights to sum to 1
3. Final: $H(x) = \text{sign}\left(\sum_{t=1}^{T} \alpha_t\, h_t(x)\right)$

Properties:

- Models trained **sequentially** (cannot parallelize)
- Models have **different weights** (better = more say)
- Reduces **BIAS**
- Can overfit if too many rounds
- Examples: AdaBoost, Gradient Boosting, XGBoost

Bagging vs Boosting — THE KEY TABLE

	Bagging	**Boosting**
Training	Independent	Sequential
Combine	Equal vote	Weighted vote
Reduces	**Variance**	**Bias**
Sampling	Bootstrap	Reweight
Overfit?	Resistant	Can overfit
Best for	Deep trees	Weak learners
Example	Random Forest	AdaBoost

6. Clustering

Setup

- **Unsupervised:** no labels, discover structure
- **Hard clustering** (this course): each point in exactly one cluster
  - $S_i \cap S_j = \emptyset$ for $i \neq j$
  - $\bigcup S_i = \{x_1, \ldots, x_n\}$
- **Soft clustering:** probabilistic membership

Hierarchical Clustering

Agglomerative = AGNES (Bottom-Up)

1. Start: each point = its own cluster (n clusters)
2. Find two **closest** clusters $\Rightarrow$ **merge**
3. Repeat until desired K
4. Produces a **dendrogram** — cut at desired level

Handles outliers well.
Linkage: Single (min), Complete (max), Average, Ward's.

Divisive = DIANA (Top-Down)

1. Start: all points in one cluster
2. Find **least cohesive** cluster $\Rightarrow$ **split**
3. Repeat until desired K

Outliers may disrupt. More expensive than agglomerative.

From Notes

All clustering is NP-Hard. We solve heuristically.

7. K-Means

Algorithm

1. Randomly select K points as centers (centroids)
2. **Assign** each point to the closest center
3. **Update** each center = mean of its assigned points
4. Repeat 2–3 until convergence (centers stop moving)

Objective Function

$$J = \sum_{i=1}^{K} \sum_{x \in S_i} \|x - \mu_i\|^2 \qquad \mu_i = \frac{1}{|S_i|} \sum_{x \in S_i} x \tag{19}$$

Equivalent: $\min \sum_{i=1}^{K} |S_i| \cdot \text{Var}(S_i)$

- **Converges?** YES (objective decreases monotonically)
- **Global optimum?** NO — local minimum only (NP-hard)
- **Fix:** run multiple times, different initializations, keep best

Choosing K: Three Methods

(i) Elbow Method: Plot J vs K. Find the "elbow" where diminishing returns begin.

(ii) Silhouette Score (per point i):

$$S(i) = \frac{b(i) - a(i)}{\max\{a(i),\ b(i)\}} \qquad S(i) \in [-1, +1] \tag{20}$$

- $a(i)$ = avg distance to **same-cluster** points (intra)
- $b(i)$ = avg distance to **nearest-other-cluster** points (inter)

- $S(i) \approx +1$: well-clustered $S(i) \approx 0$: border $S(i) \approx -1$: misclassified
- Overall score: $\bar{S} = \frac{1}{n} \sum_i S(i)$ **Higher = Better. Maximize.**

(iii) Davies-Bouldin Index:

$$DB = \frac{1}{K} \sum_{i} \max_{j \neq i} \frac{\sigma_i + \sigma_j}{\delta(c_i, c_j)} \tag{21}$$

- σ_i = intra-cluster dist (avg to centroid)
- $\delta(c_i, c_j)$ = inter-cluster dist (between centroids)

Lower DB = Better. Minimize.

8. Formula Quick Reference

Classification:

$$\text{Acc} = \frac{TP+TN}{TP+TN+FP+FN} \qquad \text{Rec} = \frac{TP}{TP+FN}$$
$$\text{Prec} = \frac{TP}{TP+FP} \qquad F_1 = \frac{2\,TP}{2\,TP+FP+FN}$$

Naïve Bayes:

$$\hat{y} = \arg\max_k P(Y{=}k) \prod_{j=1}^{d} P(X_j{=}x_j \mid Y{=}k)$$

Entropy & Info Gain:

$$H(S) = -\sum p_i \log_2 p_i$$
$$IG(S,A) = H(S) - \sum_t \frac{|S_t|}{|S|} H(S_t)$$

Gini: $\text{Gini}(S) = 1 - \sum p_i^2$

Distance:

$$d_E = \sqrt{\sum (x_j - x'_j)^2} \qquad d_M = \sum |x_j - x'_j|$$

K-Means: $J = \sum_{i=1}^{K} \sum_{x \in C_i} \|x - \mu_i\|^2$

Silhouette: $S(i) = \frac{b(i)-a(i)}{\max(a(i),b(i))} \in [-1,+1]$

Davies-Bouldin: $DB = \frac{1}{K} \sum_i \max_{j \neq i} \frac{\sigma_i + \sigma_j}{\delta(c_i,c_j)}$

AdaBoost weight: $\alpha_t = \frac{1}{2} \ln \frac{1-\epsilon_t}{\epsilon_t}$

9. Exam Traps & Tips

1. **Accuracy vs F1:** mention F1 for imbalanced data.
2. **Naïve Bayes "naïve":** conditional independence **of features given class**, not absolute.
3. **Evidence** $P(X)$ does NOT affect argmax. Ignore it.
4. **ID3 vs CART:** ID3 = multi-way + entropy. CART = binary + Gini.
5. **KNN needs scaling** (Z-score). It's a **lazy learner**.
6. **K-Means:** always converges, but to **local** minimum. Run multiple times.
7. **Silhouette:** higher = better. **DB:** lower = better.
8. **Bagging** → reduces **variance**, parallel, equal vote.
9. **Boosting** → reduces **bias**, sequential, weighted vote.
10. **1-NN:** as $n \to \infty$, error $\leq 2\times$ Bayes error.
11. **Trees are unstable:** small data changes ⇒ different tree. Fix with ensembles.
12. **AGNES** = bottom-up (handles outliers). **DIANA** = top-down (outliers disrupt).
13. **All clustering is NP-Hard** — solved heuristically.
14. **Curse of dimensionality:** NB fixes via independence; KNN has no good fix.